Μηχανική Μάθηση: Μαθηματικό Υπόβαθρο

Κωνσταντίνος Καραμανής

The University of Texas at Austin & Archimedes/Athena RC

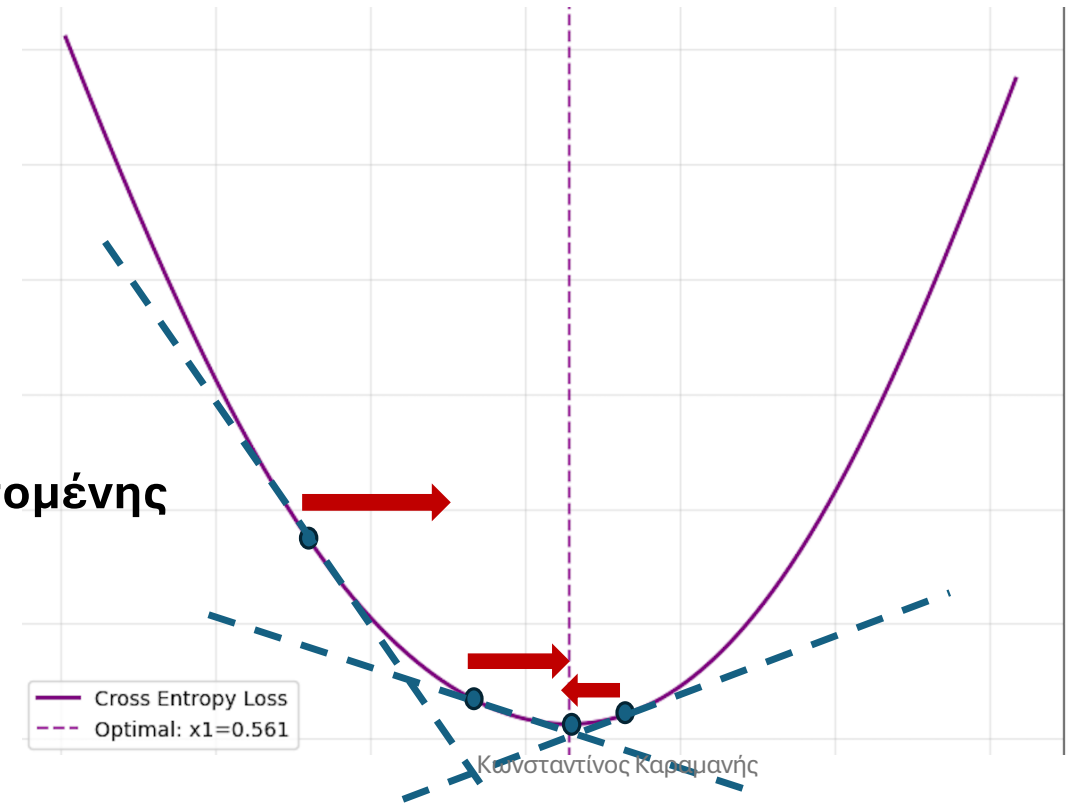
constantine@utexas.edu

https://caramanis.github.io/

Ας θυμηθούμε τα προηγούμενα...

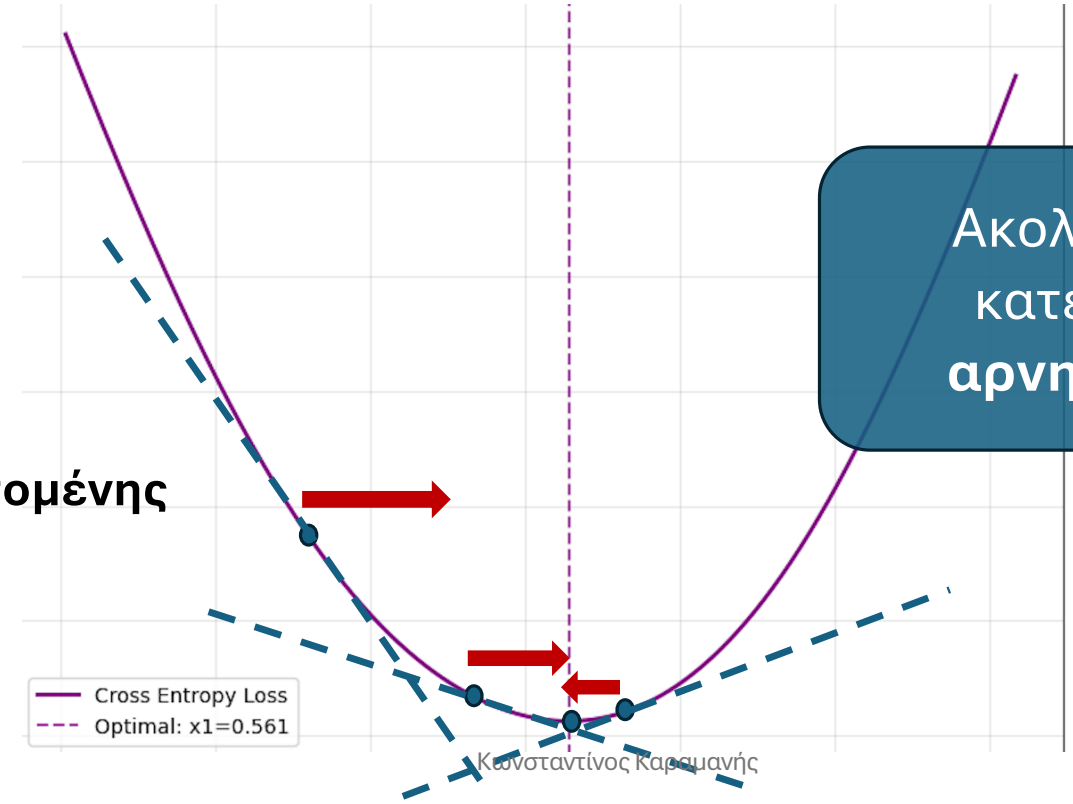
Μέθοδος: Gradient Descent (Κάθοδος Κλίσης)

κλίση της εφαπτομένης
= η **παράγωγος**



Μέθοδος: Gradient Descent (Κάθοδος Κλίσης)

κλίση της εφαπτομένης
= η **παράγωγος**



Ακολουθούμε την κατεύθυνση της **αρνητικής κλίσης**

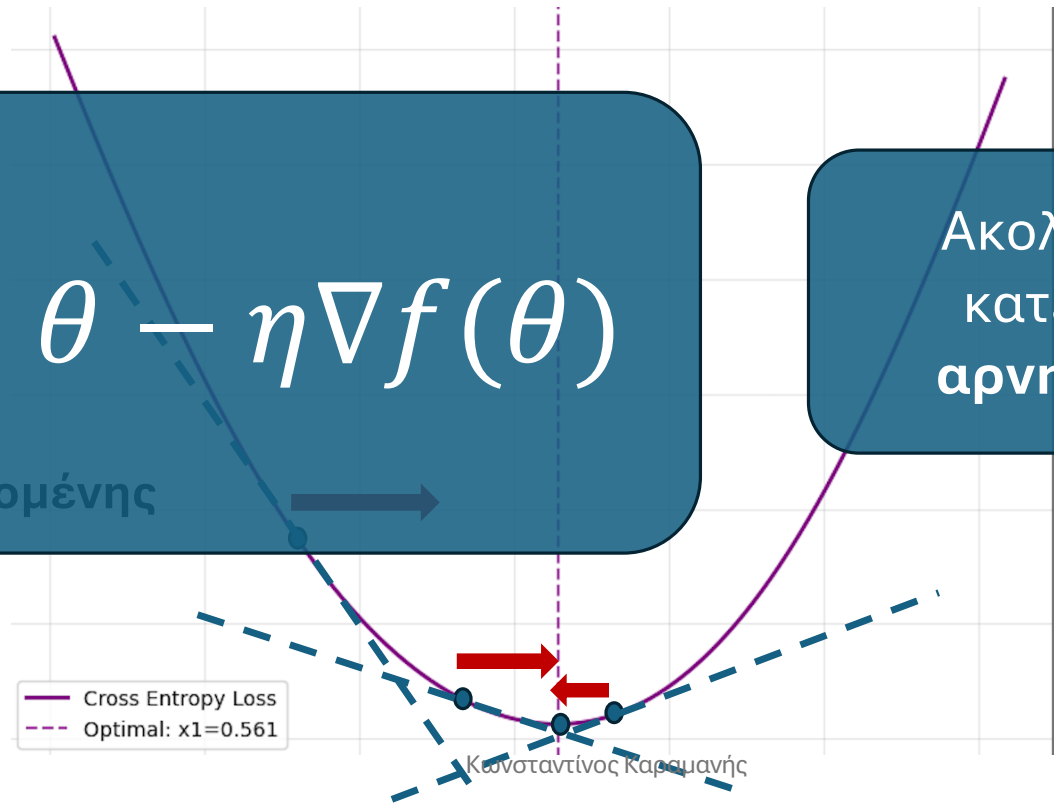
Κωνσταντίνος Καραμανής

Μέθοδος: Gradient Descent (Κάθοδος Κλίσης)

$$\theta_+ = \theta - \eta \nabla f(\theta)$$

Ακολουθούμε την κατεύθυνση της **αρνητικής κλίσης**

κλίση της εφαπτομένης
= η **παράγωγος**



Μέθοδος: Gradient Descent (Κάθοδος Κλίσης)

Παράγωγος

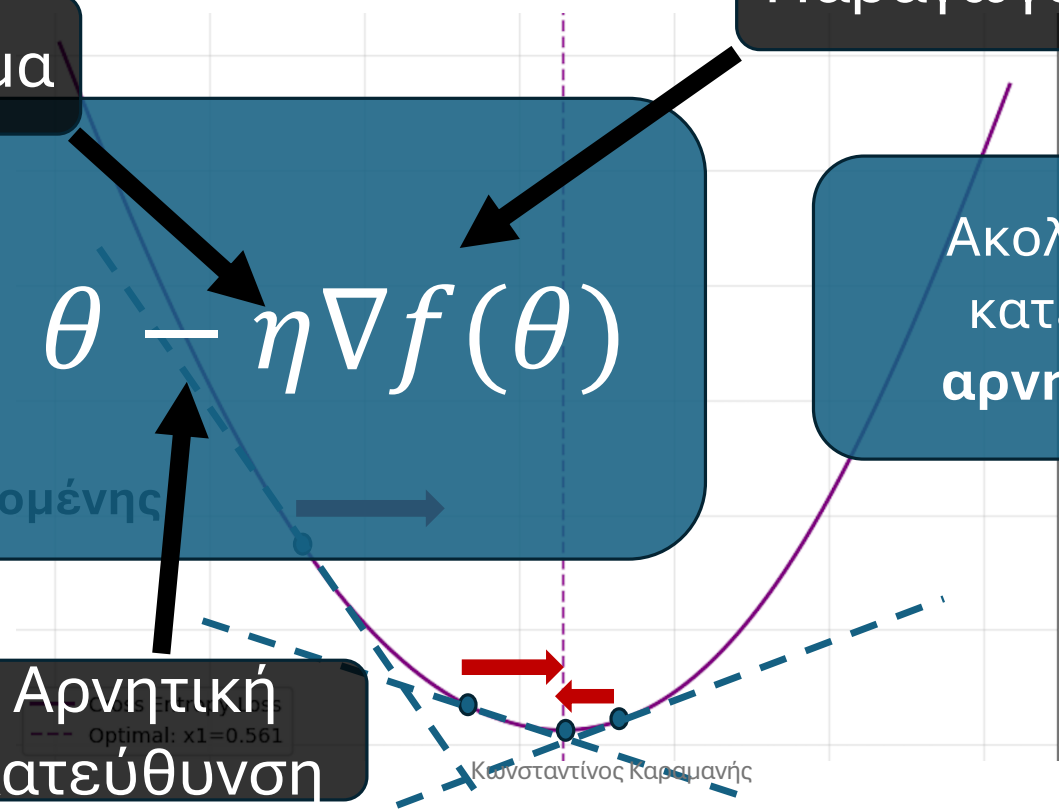
Μικρό βήμα

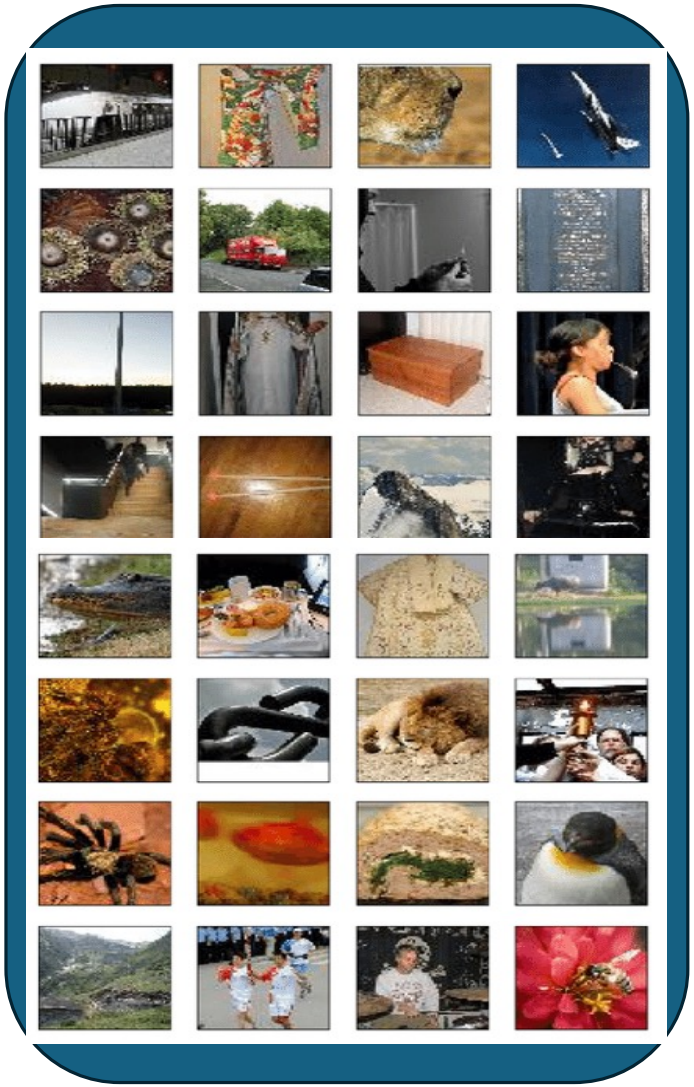
$$\theta_+ = \theta - \eta \nabla f(\theta)$$

Ακολουθούμε την κατεύθυνση της **αρνητικής κλίσης**

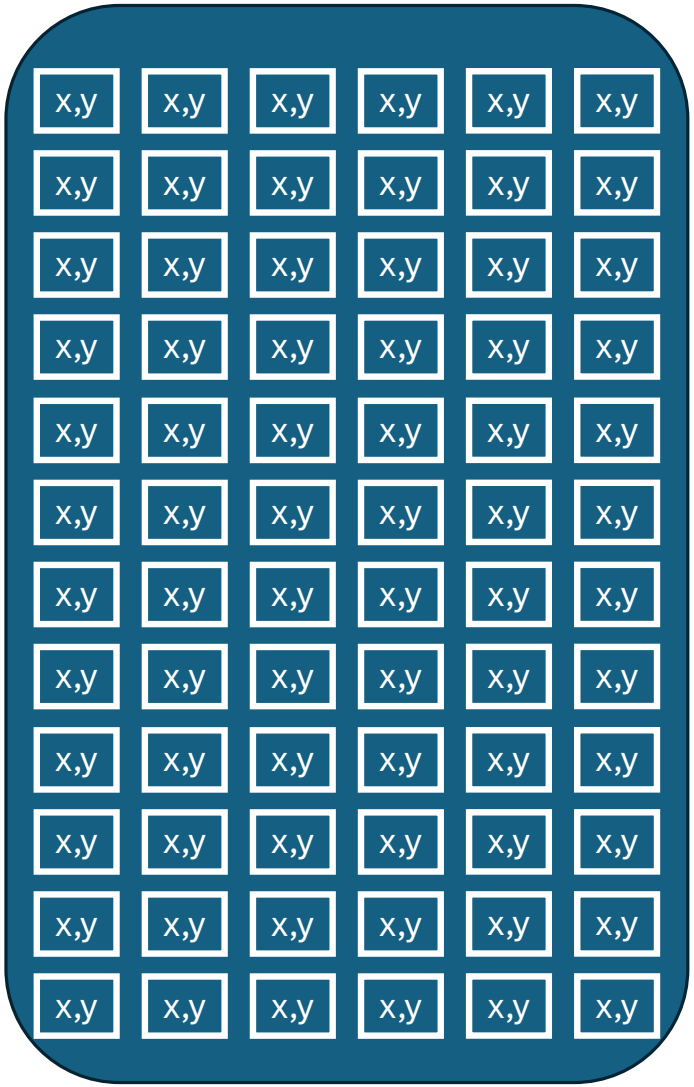
κλίση της εφαπτομένης
= η **παράγωγος**

Αρνητική κατεύθυνση

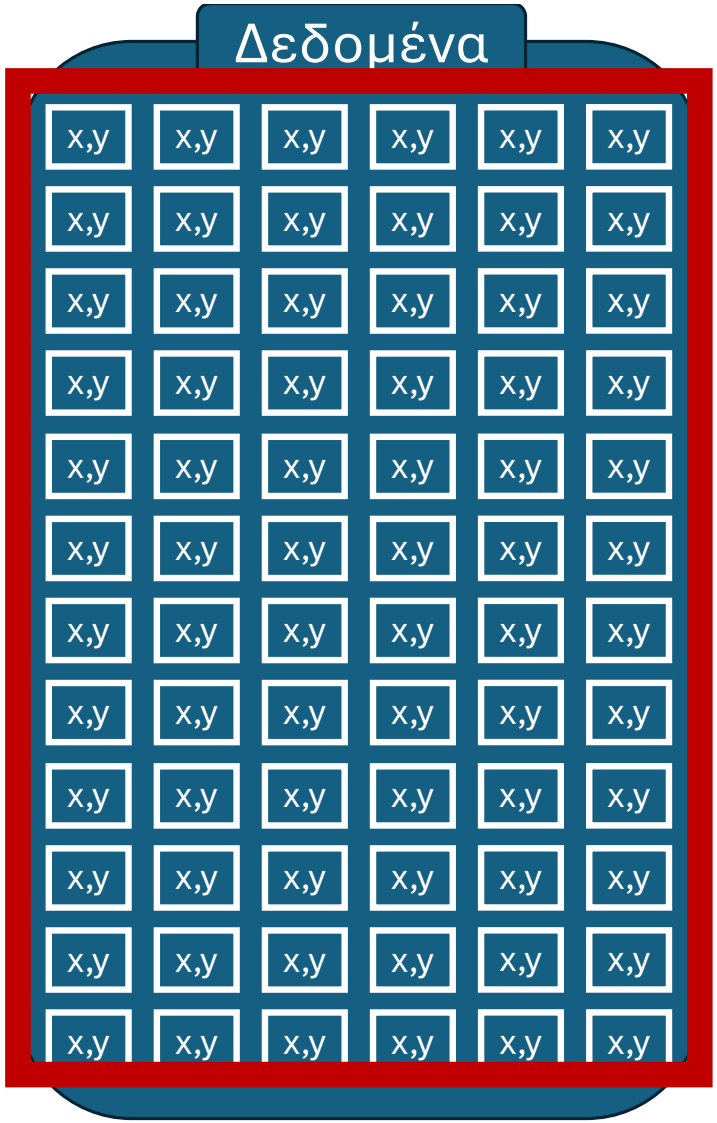


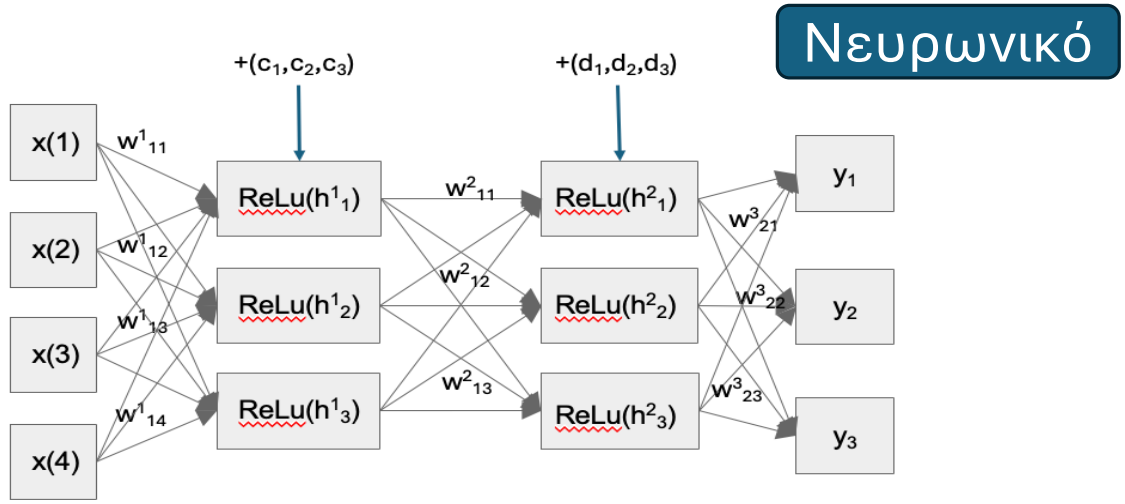


Δεδομένα



Δεδομένα





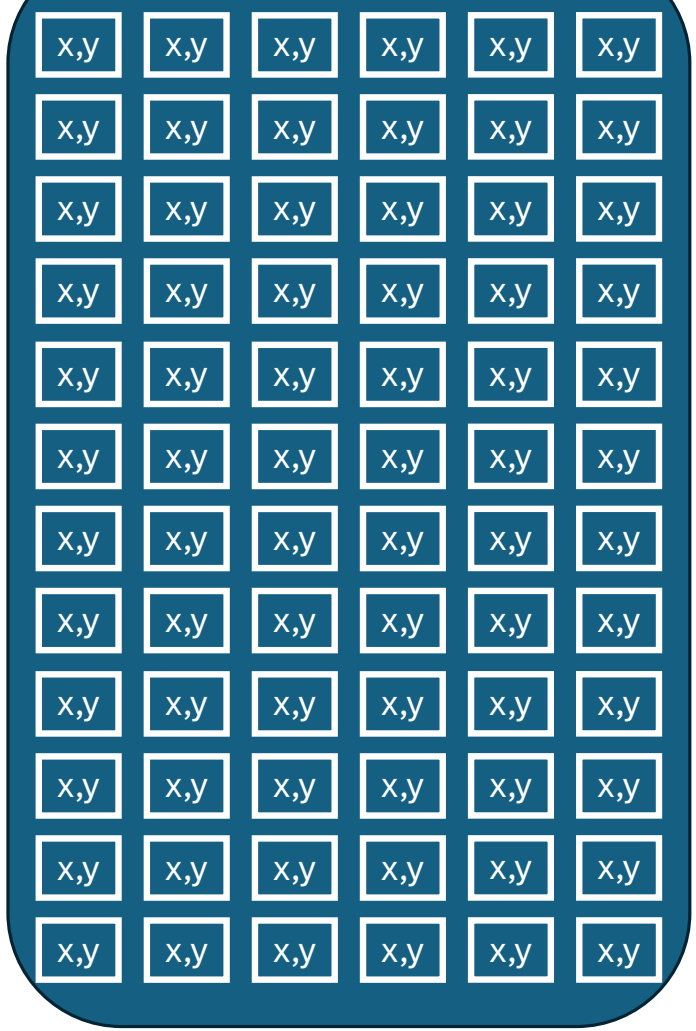
$$\theta = (W^1, c, W^2, d, W^3)$$

$$Loss(y_i, x_i, \theta) = Loss(y_i, x_i, W^1, c, W^2, d, W^3)$$

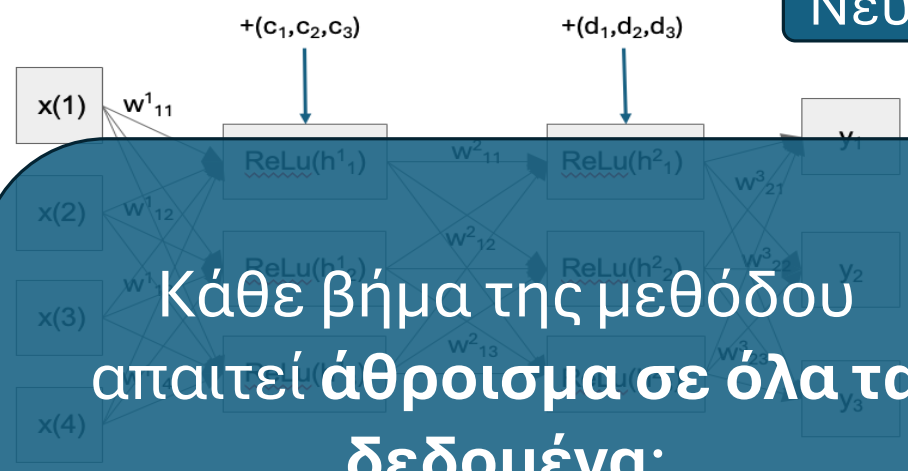
$$\mathrm{L}(\theta) = \sum Loss(y_i, x_i, \theta)$$

Συνολική απώλεια

Δεδομένα



Νευρωνικό



Κάθε βήμα της μεθόδου απαιτεί **άθροισμα σε όλα τα δεδομένα:**

$$\theta_+ = \theta - \eta \nabla L(\theta)$$

$$= \theta - \eta \sum_i \nabla Loss(x_i, y_i, \theta)$$

Συνολική απώλεια

Δεδομένα

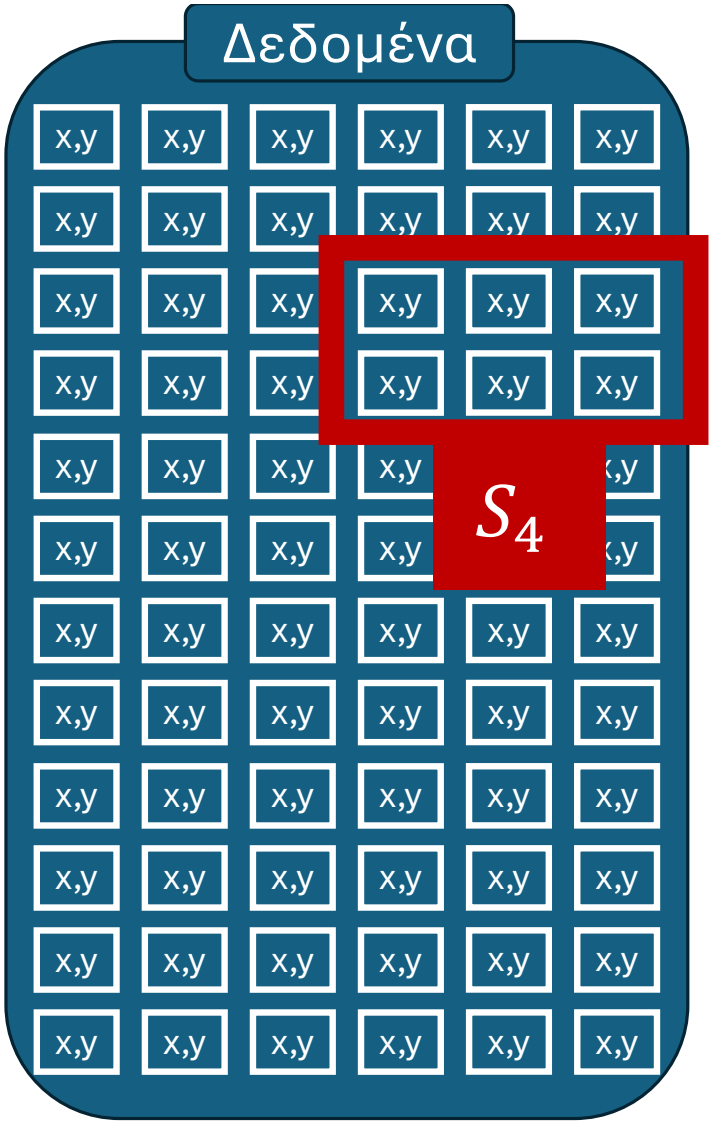
Νευρωνικό

Κάθε βήμα της μεθόδου

Gradient Descent + Minibatch (`batch_size`)

→ Stochastic Gradient Descent

Συνολική απώλεια



$$\theta_{t+1} = \theta_t - \eta \sum_{i \in S_1} \nabla Loss(x_i, y_i, \theta_t)$$

$$\theta_{t+2} = \theta_{t+1} - \eta \sum_{i \in S_2} \nabla Loss(x_i, y_i, \theta_{t+1})$$

$$\theta_{t+3} = \theta_{t+2} - \eta \sum_{i \in S_3} \nabla Loss(x_i, y_i, \theta_{t+2})$$

$$\theta_{t+4} = \theta_{t+3} - \eta \sum_{i \in S_3} \nabla Loss(x_i, y_i, \theta_{t+3})$$

Η Μέθοδος *Stochastic* Gradient Descent σε κώδικα Python

```python
# dataloaders
train_loader = DataLoader(train_dataset, batch_size=64,
                          shuffle=True, num_workers=2)
```

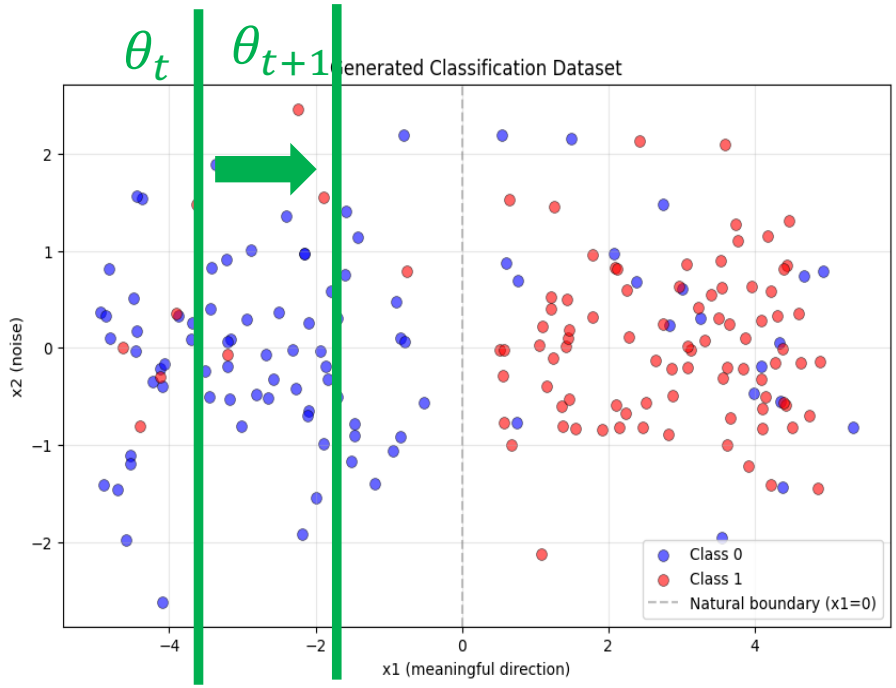
```python
for epoch in range(epochs):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

Η Μέθοδος: Stochastic Gradient Descent

Η συνολική απώλεια:

$$\sum Loss(x_i, y_i, \theta)$$

Είναι συνάρτηση των **παραμέτρων** και των δεδομένων.



Όταν **εκπαιδεύουμε** βρίσκουμε παραμέτρους που ελαχιστοποιούν την απώλεια

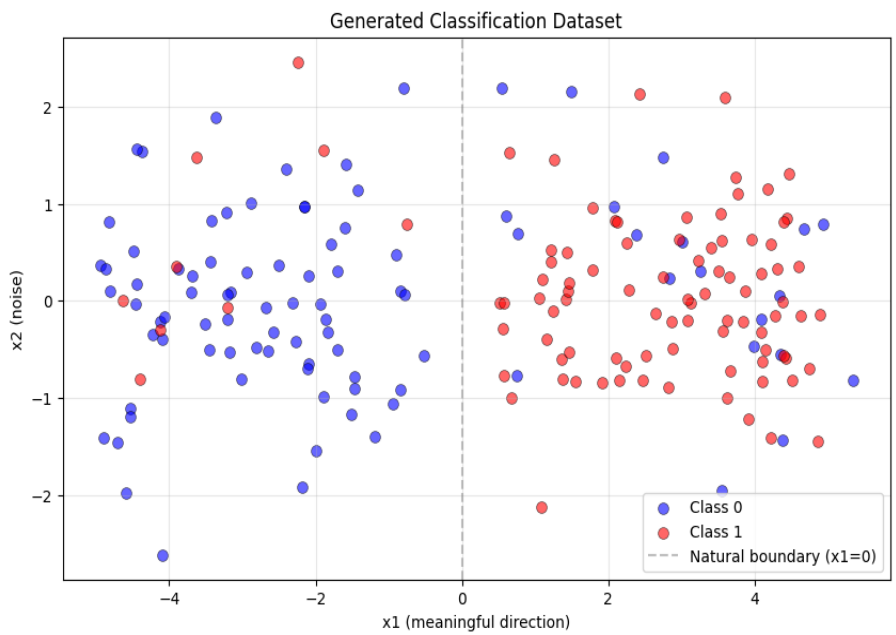
Stochastic Gradient Descent

$$\theta_+ = \theta - \eta \sum_{i \in S} \nabla Loss(x_i, y_i, \theta)$$

Η συνολική απώλεια:

$$\sum Loss(x_i, y_i, \theta)$$

Είναι συνάρτηση των παραμέτρων και των δεδομένων.



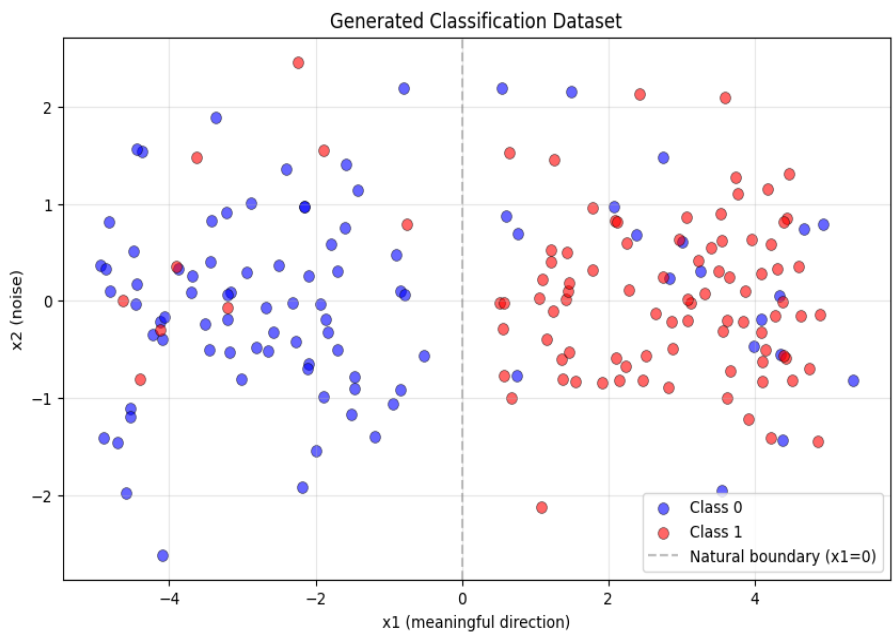
Πότε θα θέλαμε να μετακινήσουμε ένα σημείο ώστε να αυξήσουμε ή να μειώσουμε την απώλεια του σημείου χωρίς να πειράξουμε/αλλάξουμε το νευρωνικό δίκτυο;

$$x_+ = x \pm \eta \nabla_x Loss(x, \theta)$$

Η συνολική απώλεια:

$$\sum Loss(x_i, y_i, \theta)$$

Είναι συνάρτηση των παραμέτρων και των δεδομένων.



Η συνολική απώλεια:

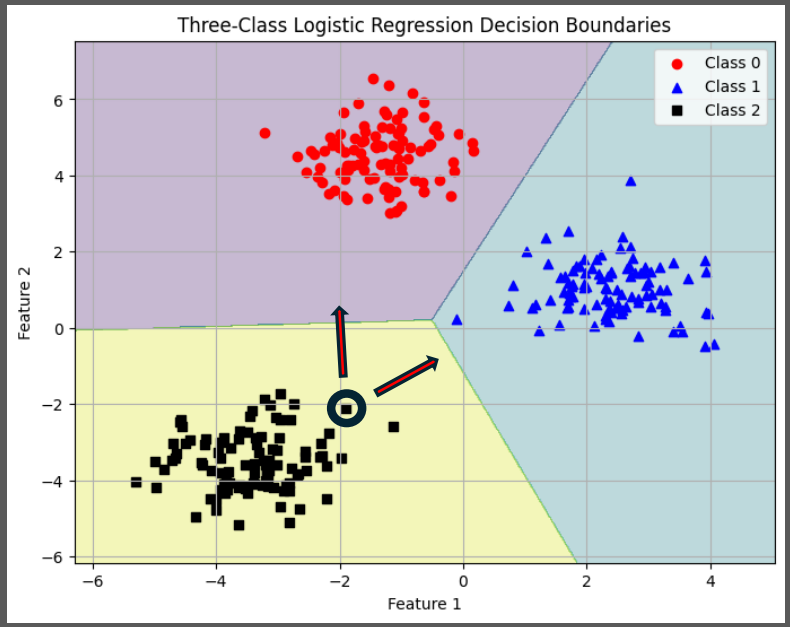
Κακόβουλη επίθεση ενάντια στο νευρωνικό δίκτυο

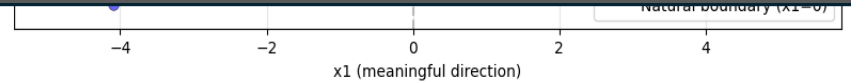
“Adversarial Attacks”

x – μαύρο σημείο

$$x^{+} = x - \eta \nabla_x L(x, y, \theta)$$

$$x^{+} = x - \eta \nabla_x L(x, y, \theta)$$



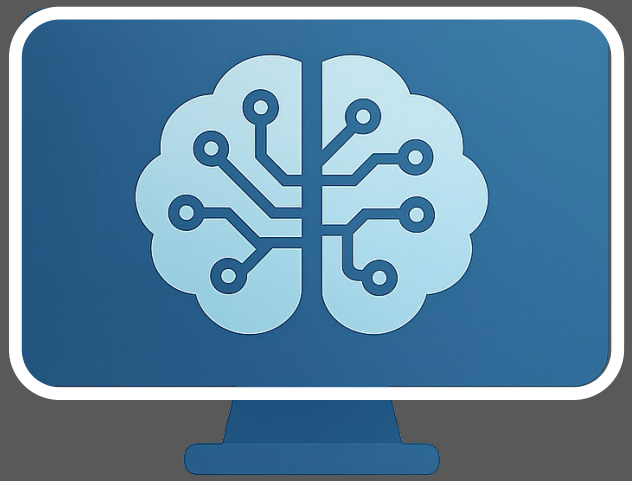


Η συνολική απώλεια:

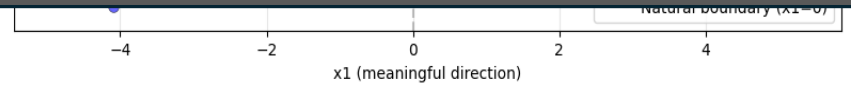
Κακόβουλη επίθεση ενάντια στο νευρωνικό δίκτυο

“Adversarial Attacks”

resnet18



Golden retriever: 50%
Dog: 99.96%

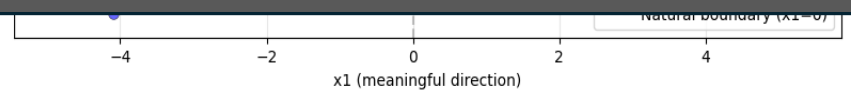


Η συνολική απώλεια:

Κακόβουλη επίθεση ενάντια στο νευρωνικό δίκτυο

“Adversarial Attacks”

$$-\ \eta \nabla_x L(x_{\text{dog}}, y_{\text{cat}}, \theta)$$

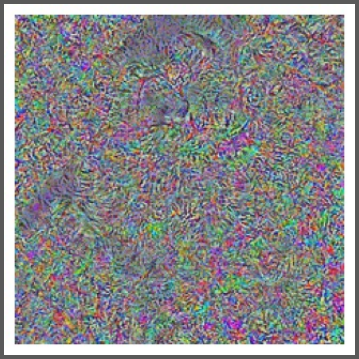


Η συνολική απώλεια:

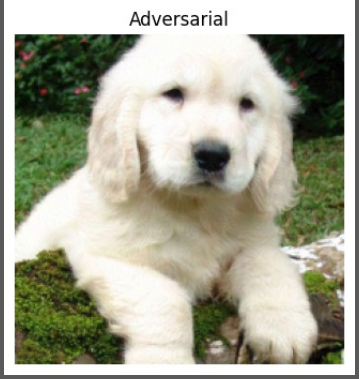
Κακόβουλη επίθεση ενάντια στο νευρωνικό δίκτυο

“Adversarial Attacks”

−



=

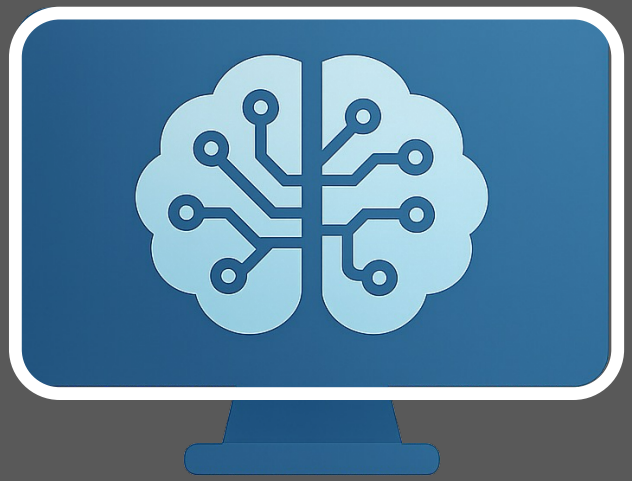


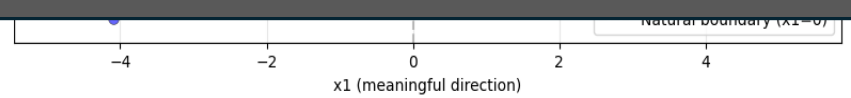
Η συνολική απώλεια:

Κακόβουλη επίθεση ενάντια στο νευρωνικό δίκτυο

“Adversarial Attacks”

resnet18



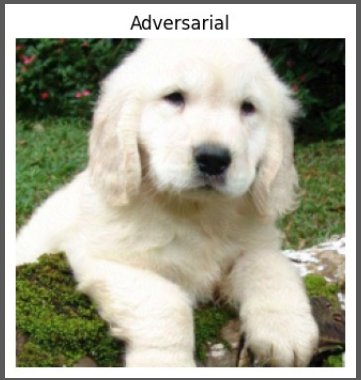


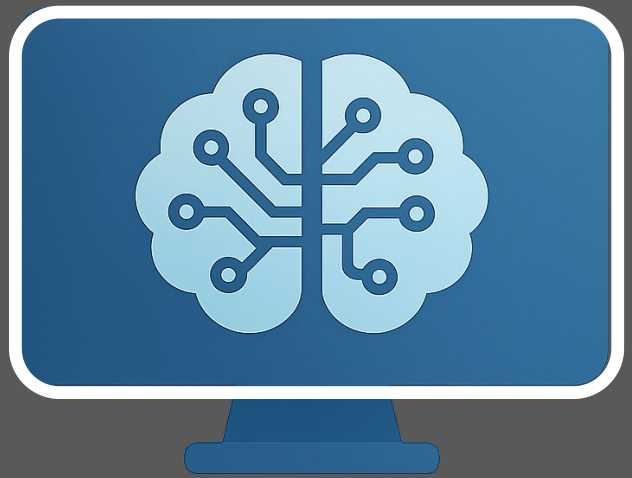
Η συνολική απώλεια:

Κακόβουλη επίθεση ενάντια στο νευρωνικό δίκτυο

“Adversarial Attacks”

resnet18



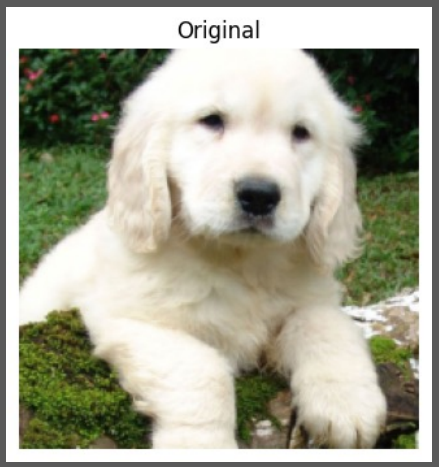


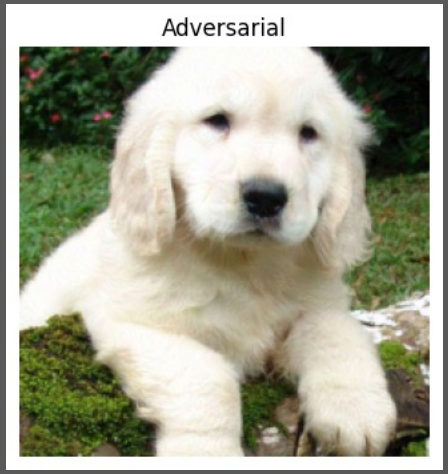
Siamese Cat: 100%

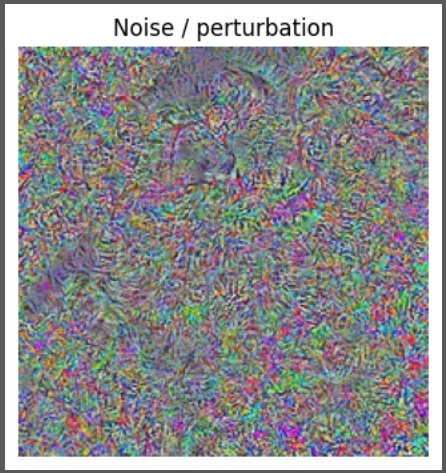
Η συνολική απώλεια:

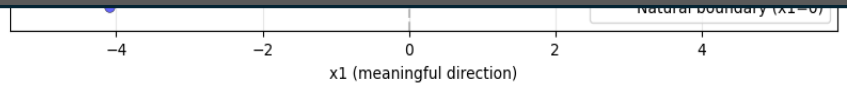
Κακόβουλη επίθεση ενάντια στο νευρωνικό δίκτυο

“Adversarial Attacks”









Εκπαίδευση: Αλλάζουμε το μοντέλο (θ)

Stochastic Gradient Descent

$$\theta_+ = \theta - \eta \sum_{i \in S} \nabla Loss(x_i, y_i, \theta)$$

Επίθεση: Αλλάζουμε το σημείο (x)

$$x_+ = x \pm \eta \nabla_{\mathrm{x}} Loss(x, y, \theta)$$

Η Μέθοδος: Stochastic Gradient Descent

Βασικές Παραλλαγές της Μεθόδου

Gradient Descent

$$\theta_+ = \theta - \eta \nabla L(\theta)$$

Stochastic Gradient Descent

$$\theta_+ = \theta - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta)$$

SGD + Momentum

$$\theta_{t+1} = \theta_t - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha(\theta_t - \theta_{t-1})$$

Adaptive SGD + Momentum (ADAM)

$$\theta_{t+1} = \theta_t - \eta_t \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha_t(\theta_t - \theta_{t-1})$$

Gradient Descent: σε κώδικα και στην πράξη

Κωνσταντίνος Καραμανής

Stochastic Gradient Descent

$$\theta_+ = \theta - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta)$$

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
```

```python
criterion = torch.nn.CrossEntropyLoss()
```

```python
# optimizer definition
optimizer = optim.SGD(model.parameters(), lr=0.0035)
```

```python
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

```python
# dataloader
train_loader = DataLoader(train_dataset, batch_size=256,
                          shuffle=True)
```

Οπότε γιατί θα επιλέγαμε ποτέ να τρέξουμε SGD

Gradient Desc

$$\theta_+ = \theta - \eta \nabla L$$

vs

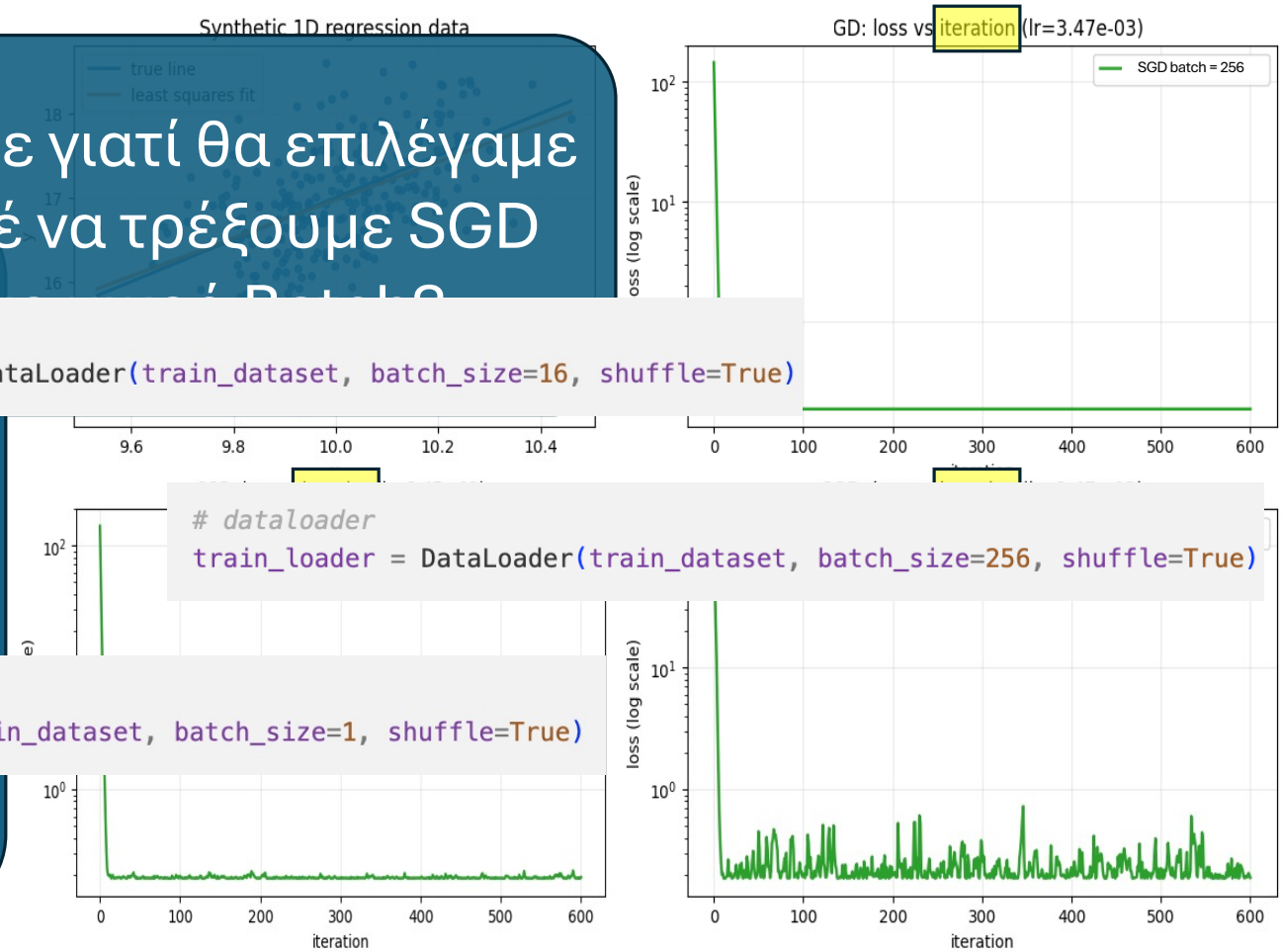
Stochastic Gradient

$$\theta_+ = \theta - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta)$$

```
# dataloader
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
```

```
# dataloader
train_loader = DataLoader(train_dataset, batch_size=256, shuffle=True)
```

```
# dataloader
train_loader = DataLoader(train_dataset, batch_size=1, shuffle=True)
```



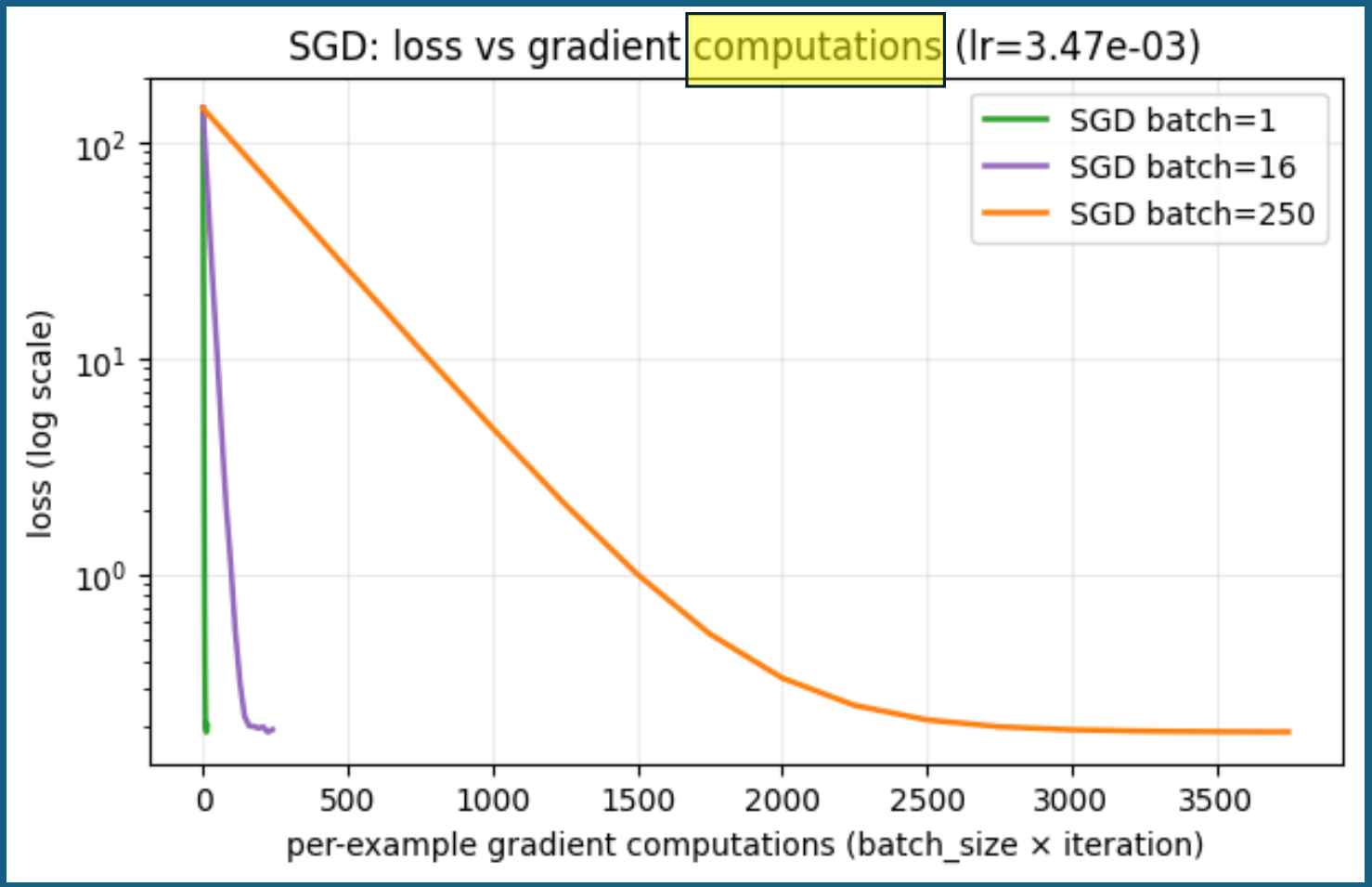
Grad

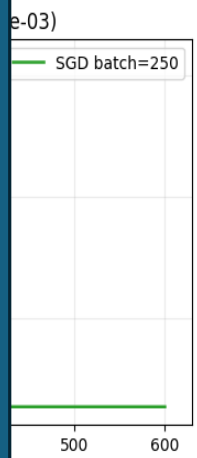
$\theta_+ = \theta$

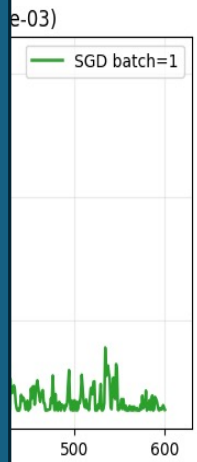
Stocha

D

$\theta_+ = \theta - \eta$







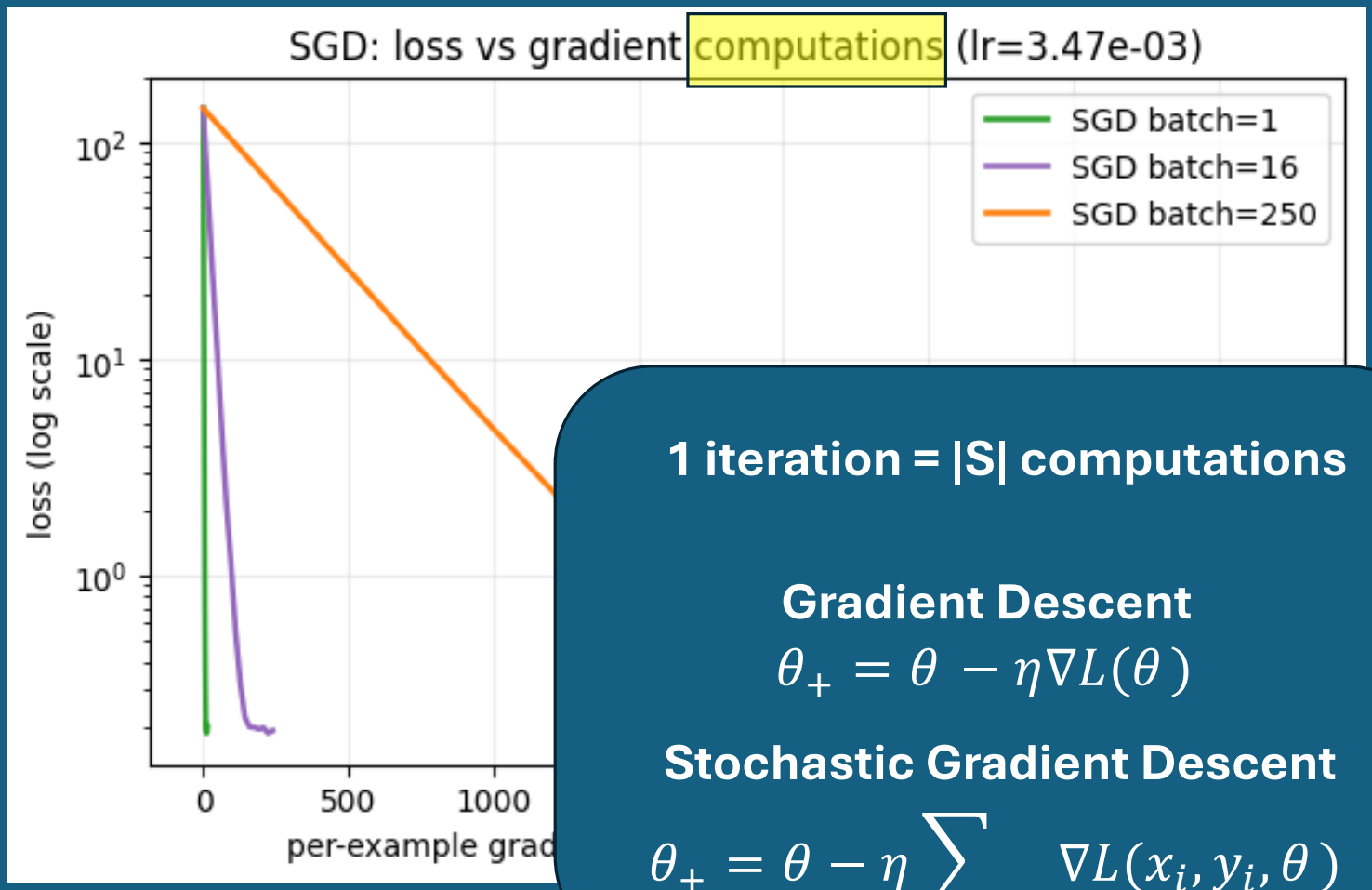
Grad

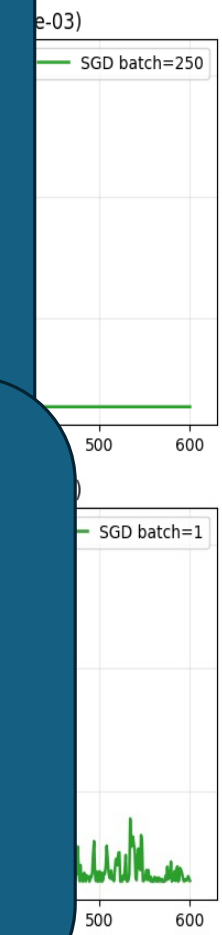
$\theta_+ = \theta$

Stocha

D

$\theta_+ = \theta - \eta$





1 iteration = |S| computations

Gradient Descent

$$\theta_+ = \theta - \eta \nabla L(\theta)$$

Stochastic Gradient Descent

$$\theta_+ = \theta - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta)$$

SGD + Momentum

$$\theta_{t+1} = \theta_t - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha(\theta_t - \theta_{t-1})$$

SGD + Momentum

$$\theta_{t+1} = \theta_t - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha(\theta_t - \theta_{t-1})$$

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
```

```python
criterion = torch.nn.CrossEntropyLoss()
```

```python
# optimizer definition
optimizer = optim.SGD(model.parameters(), lr=0.001,
                      momentum=0.9)
```

```python
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

```python
# dataloader
train_loader = DataLoader(train_dataset, batch_size=256,
                          shuffle=True)
```

SGD + Momentum

$$\theta_{t+1} = \theta_t - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha(\theta_t - \theta_{t-1})$$

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
```

```python
criterion = torch.nn.CrossEntropyLoss()
```

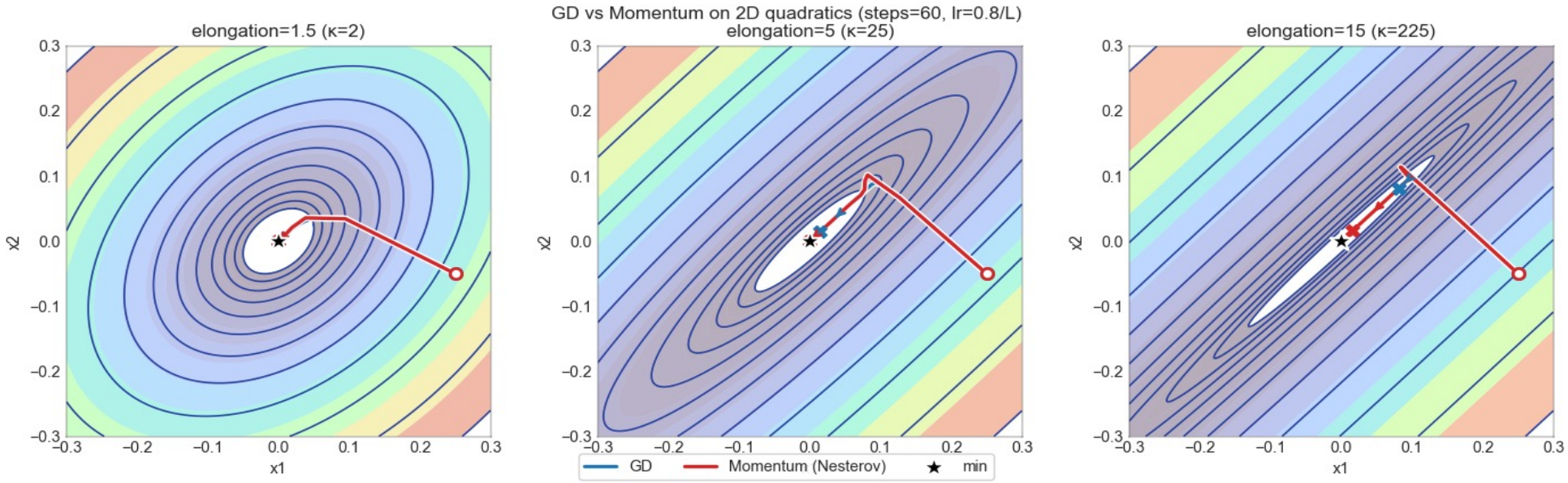
```python
# optimizer definition
optimizer = optim.SGD(model.parameters(), lr=0.001,
                      momentum=0.9)
```

```python
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

```python
# dataloader
train_loader = DataLoader(train_dataset, batch_size=256,
                          shuffle=True)
```

SGD + Momentum

$$\theta_{t+1} = \theta_t - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha(\theta_t - \theta_{t-1})$$

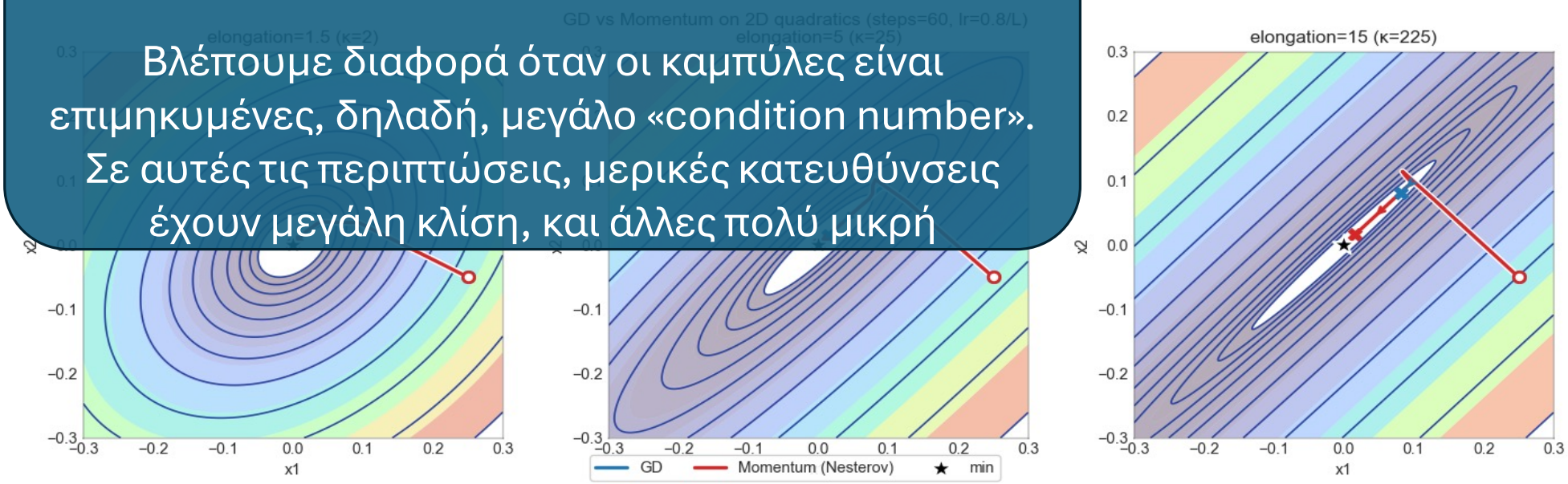


SGD + Momentum

$$\theta_{t+1} = \theta_t - \eta \sum \nabla L(x_i, y_i, \theta_t) + \alpha(\theta_t - \theta_{t-1})$$

Όταν είναι κυκλικές οι ισοϋψείς καμπύλες το «momentum» δεν προσφέρει μεγάλο πλεονέκτημα.

Βλέπουμε διαφορά όταν οι καμπύλες είναι επιμηκυμένες, δηλαδή, μεγάλο «condition number». Σε αυτές τις περιπτώσεις, μερικές κατευθύνσεις έχουν μεγάλη κλίση, και άλλες πολύ μικρή



Adaptive SGD + Momentum (ADAM)

$$\theta_{t+1} = \theta_t - \eta_t \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha_t(\theta_t - \theta_{t-1})$$

```
# optimizer definition
optimizer = optim.Adam(model.parameters(), lr=0.0035)
```

η_t: συνάρτηση των προσφάτων $||\nabla L(\theta_t)||$

Μεγαλώνει το βήμα (learning rate, lr) η_t εάν έχουμε πάρει πολλά μικρά βήματα

Μικραίνει το βήμα (learning rate, lr) η_t εάν έχουμε πάρει πολλά μεγάλα βήματα

Η Μέθοδος: Stochastic Gradient Descent

Βασικές Παραλλαγές της Μεθόδου

Gradient Descent

$$\theta_+ = \theta - \eta \nabla L(\theta)$$

Stochastic Gradient Descent

$$\theta_+ = \theta - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta)$$

SGD + Momentum

$$\theta_{t+1} = \theta_t - \eta \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha(\theta_t - \theta_{t-1})$$

Adaptive SGD + Momentum (ADAM)

$$\theta_{t+1} = \theta_t - \eta_t \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha_t(\theta_t - \theta_{t-1})$$

Η Μέθοδος:
Stoch
Gradie
Desce

Βασικές Παραλλαγές της Μεθόδου

Gradient Descent

$$\theta_+ = \theta - \eta \nabla L(\theta)$$

Stochastic Gradient Descent

$$x_i, y_i, \theta)$$

m

$$\theta_t) + \alpha(\theta_t - \theta_{t-1})$$

$$\sum_{i \in S}$$

Σας προτείνω να δομικάσ(ζ)ετε και εσείς τις διάφορες παραλλαγές τις μεθόδου σε προβλήματα όπου έχετε την υπολογιστική δυνατότητα να κάνετε πειράματα.

Adaptive SGD + Momentum (ADAM)

$$\theta_{t+1} = \theta_t - \eta_t \sum_{i \in S} \nabla L(x_i, y_i, \theta_t) + \alpha_t(\theta_t - \theta_{t-1})$$